ПРОЕКТ «АНТИФРОД» · НЕДЕЛЯ 5 · КОМАНДА ИЗ ТРЁХ УЧАСТНИЦ

Антифрод + PostgreSQL

Журнал решений переехал из SQLite и памяти в отдельный контейнер с базой. Данные переживают перезапуск, метрики считаются SQL-запросами, падение базы не роняет сервис.

A · схема

«контейнеры и запуск»

Назлы

B · таблицы

«что и как пишется»

Лия

C · надёжность

«отказ базы и бонусы»

Шырайлым

3 таблицы

request_log · transactions · labels — создаются сами из init.sql

0 потерь

после docker compose down / up: числа на панели те же

503 за 0,15 с

если база упала: сервис отвечает честно и сам восстанавливается

говорит: A

Схема: пять контейнеров, данные — в томе pgdata

docker compose up -d --build поднимает всё сразу; контейнеры общаются по именам сервисов

Интерфейс оператора · :8501

frontend — ввод операции, серия, сверка с меткой

Панель мониторинга · :8502

dashboard — показатели из /metrics, окно 5 мин / час / всё

Adminer · :8080 (доп.)

просмотр таблиц базы в браузере

▼ HTTP · http://backend:8000 ▼

backend · FastAPI :8000

модель v2 + история карт в памяти; каждый запрос → middleware → запись в базу

`POST /predict` `POST /label`

`GET /metrics` `GET /health`

Внутренняя сеть Docker

frontend и dashboard ходят на backend:8000, backend и Adminer — на db:5432

▼ SQL · postgresql+psycopg://…@db:5432 ▼

db · postgres:16

init.sql создаёт 3 таблицы при первом старте; healthcheck pg_isready; снаружи — порт 5433

том pgdata

файлы базы живут здесь, а не в контейнере → переживают down / up

.env (не в git)

POSTGRES_USER, POSTGRES_PASSWORD; в репозитории только .env.example

Порядок старта: db (healthy) → backend (healthy) → frontend, dashboard. Интерфейс и панель с базой не говорят — только через backend.

docker-compose.yml · db/init.sql · .env.example · README.md, раздел «Неделя 5»

говорит: А

Запуск, пароль и сохранность данных

То, что проверяющий делает первым делом

Команда	Что происходит
`zapusk_docker.bat`	.env из .env.example → up -d --build → ps
`docker compose ps`	db (healthy), backend (healthy), frontend, dashboard, adminer
`docker compose down`	контейнеры удалены, том pgdata остаётся
`docker compose up -d`	данные на месте, панель показывает те же числа
`docker compose down -v`	удаляет и том — база с нуля (осторожно)
`psql_db.bat`	psql внутри контейнера db

```
docker compose exec db psql -U antifraud -d antifraud
antifraud=# SELECT count(*) FROM transactions;
antifraud=# SELECT status_code, error FROM request_log
            WHERE status_code = 422 ORDER BY ts DESC LIMIT 3;
```

Пароль

.env в .gitignore и .dockerignore; в репозитории .env.example с change_me. compose берёт пароль из .env и для базы, и для строки подключения backend.

Ожидание базы

healthcheck db: pg_isready каждые 5 с; backend: depends_on db condition: service_healthy — не стартует, пока база не готова.

Порт 5433

снаружи база на 5433, чтобы не конфликтовать с локальным Postgres; внутри сети — db:5432.

docker-compose.yml · zapusk_docker.bat · psql_db.bat · .gitignore

говорит: А

Три таблицы: запрос, решение, метка

db/init.sql — Postgres выполняет его сам, когда том pgdata пустой

request_log

каждый HTTP-запрос

```
id  BIGSERIAL
ts  TIMESTAMPTZ
endpoint  /predict…
method
status_code  200/422/503
error  причина
latency_ms
```

transactions

каждое решение модели

```
id  UUID = transaction_id
received_at
card_id, amount, category, merchant
payload  JSONB — вся операция
fraud_score, is_fraud_pred
threshold, risk_level
model_version, latency_ms
```

labels

факт, пришёл позже

```
transaction_id  UUID
  → REFERENCES transactions
is_fraud  CHECK 0 / 1
labeled_at
```

Индексы под /metrics

```
request_log (ts), (endpoint, ts)
transactions (received_at), (card_id)
(received_at DESC) WHERE is_fraud_pred = 1
```

Почему так

запрос ≠ решение: 422 есть в request_log, но не в transactions. Метка — отдельно, потому что приходит позже; внешний ключ не даст пометить несуществующую операцию (→ 404).

говорит: В

Как сервис пишет в базу и считает /metrics

Все SQL-запросы параметризованные (SQLAlchemy + psycopg 3), без склейки строк

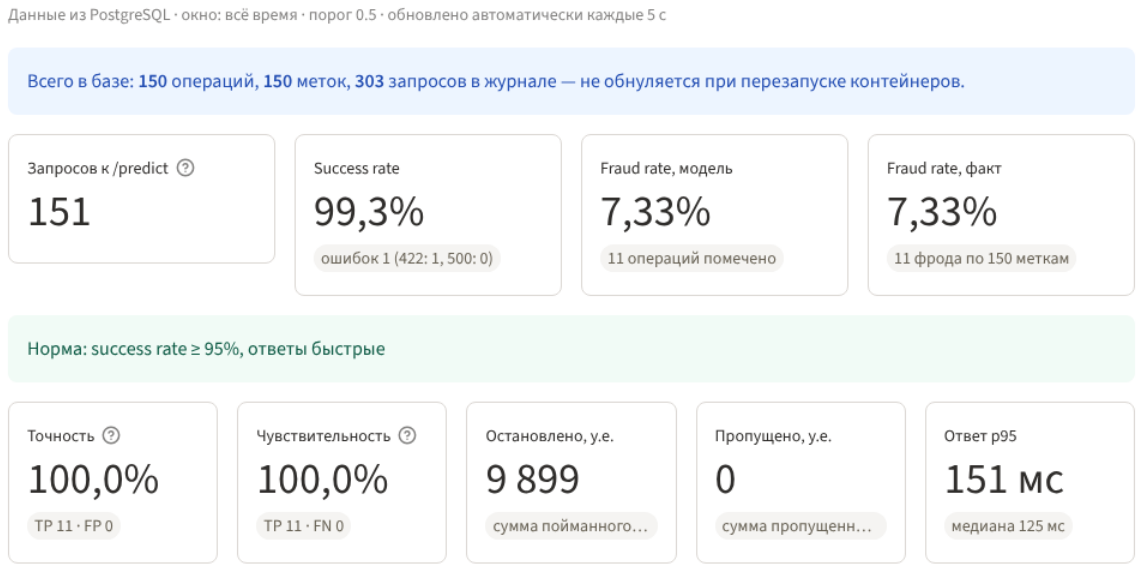
middleware — любой запрос, даже 422 до функции predict → request_log

POST /predict — оценка → INSERT transactions → только потом история карты

POST /label — INSERT … ON CONFLICT DO UPDATE в labels; нет операции → 404

GET /metrics — SQL по окну ?window=5m | 1h | all

```
SELECT avg((status_code < 400)::int)
         AS success_rate,
       count(*) FILTER (WHERE status_code=422)
FROM request_log
WHERE endpoint = '/predict'
  AND ts > now() - interval '5 minutes';
```



Панель: «Всего в базе: 150 операций, 150 меток, 303 запроса»

Что даёт SQL вместо Python

success rate, fraud rate (модель и факт), TP / FP / FN, p95 задержки, график по минутам — одним запросом к базе; окно меняется параметром, числа не обнуляются при перезапуске.

service/main.py (middleware, /predict, /label) · service/db.py (compute_metrics)

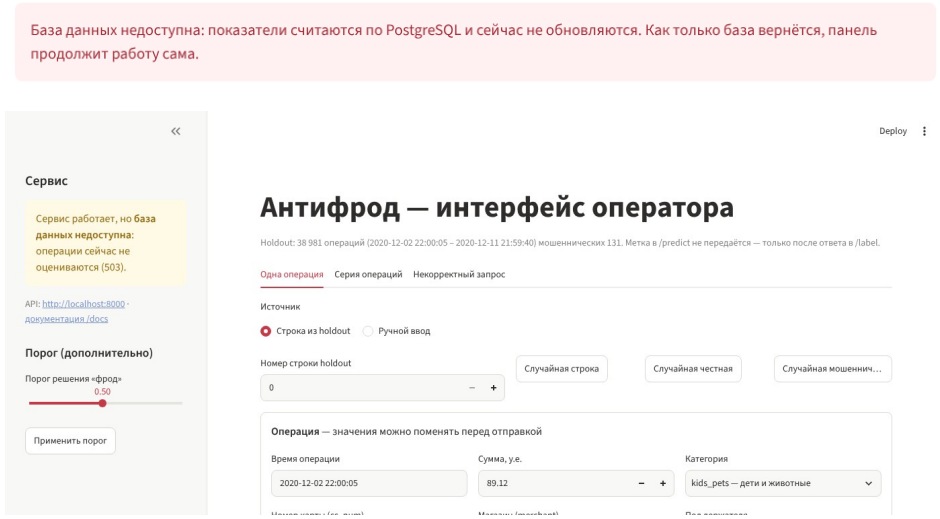
говорит: В

База упала — сервис не падает

docker compose stop db → честный ответ; start db → работает сам, без перезапуска backend

	База работает	База остановлена
GET /health	200 · status: ok · db: up	200 · status: degraded · db: down
POST /predict	200 · решение записано	503 · «операция не оценена»
POST /label	200 / 404	503
GET /metrics	200 · SQL	503
request_log	пишется	пропуск, сервис не падает
интерфейс	результат	жёлтое предупреждение
панель	показатели	«База данных недоступна…»

Антифрод — панель мониторинга



Почему сразу 503

не оцениваем «вслепую»: у антифрода должен быть след каждого решения, иначе его нельзя проверить

Как восстанавливается

pool_pre_ping проверяет соединение перед запросом; connect_timeout 3 с — ответ 503 за 0,15 с

Проверили

stop db → 503; start db → следующий /predict 200; рестарт backend — история карт из базы

service/db.py (DatabaseUnavailable) · service/main.py (/health) · frontend · dashboard

говорит: С

Дополнительно и план демонстрации

Adminer

http://localhost:8080 — таблицы и SQL в браузере

Индексы + EXPLAIN

окно 5 мин по 300 тыс. строк: 0,19 мс с индексом против 23,7 мс без

Резервная копия

backup_db.bat → pg_dump;
restore_db.bat → pg_restore

История карт в Postgres

при старте backend догружает операции из transactions.payload

Не сделали и почему

Alembic — схема пока одна, хватает init.sql
init.sql работает только на пустом томе: новая колонка = down -v или ALTER TABLE
Одна копия backend — история карт в памяти процесса

Демонстрация · 5 минут

1. up -d --build → ps: db healthy
2. интерфейс: серия 100 операций
3. psql: SELECT count(*) FROM transactions
4. некорректный запрос → 422 в request_log
5. down → up → панель: числа те же
6. stop db → /health degraded, /predict 503
7. start db → снова 200, без перезапуска
8. SQL-запрос от преподавателя

README.md, разделы «Неделя 5», «Дополнительно» · db/queries.sql

говорит: С